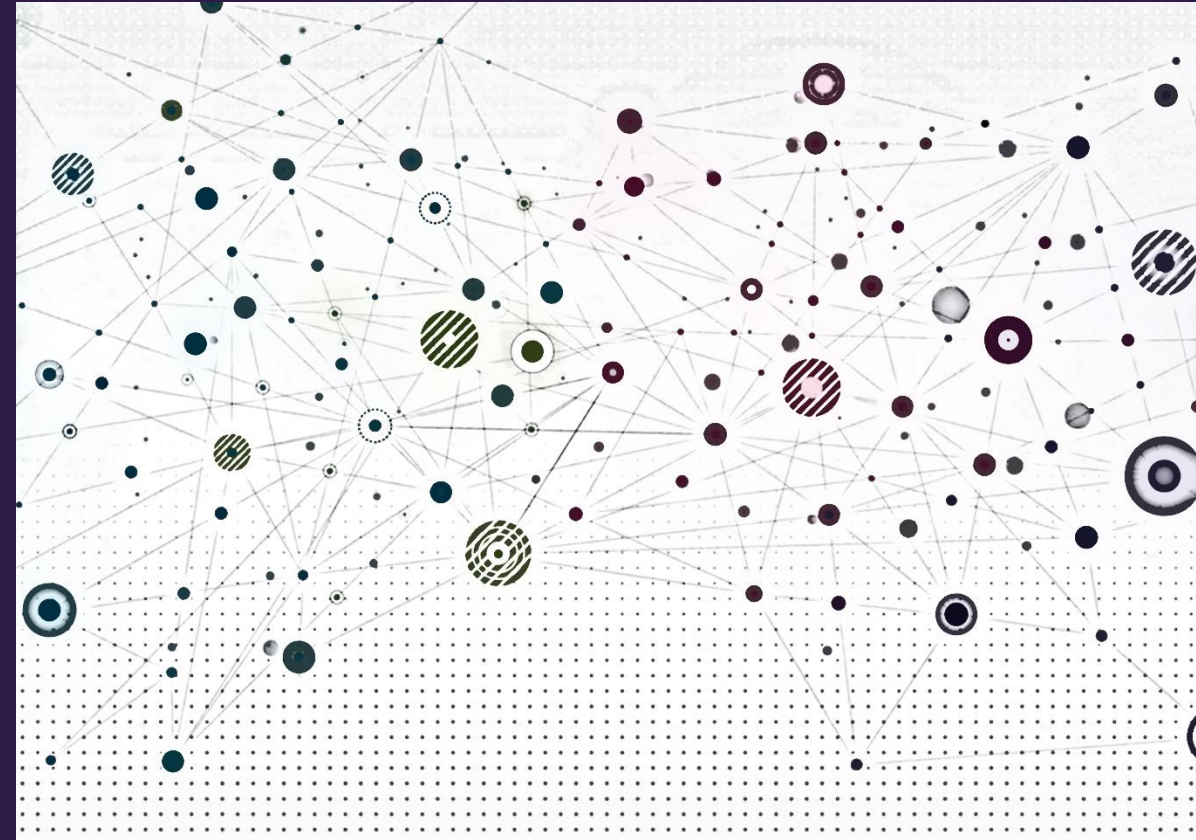


In-context learning over graphs





Overview

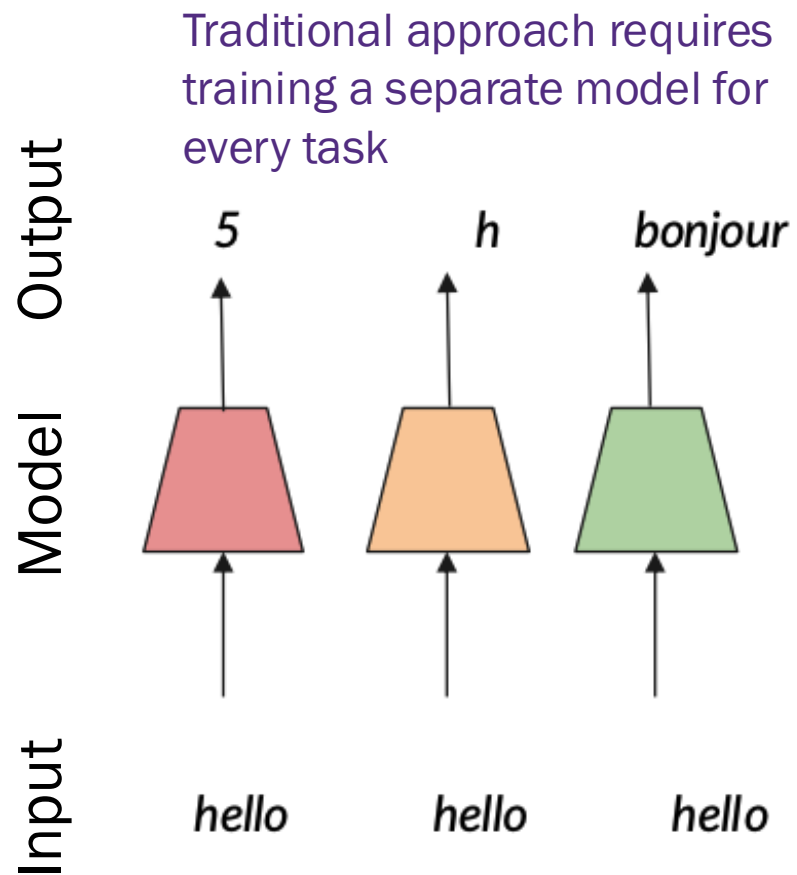
- What is in-context learning?
- In-context learning over graphs formulation
- Pretraining Over Diverse In-Context Graph Systems (PRODIGY)



What is in-context learning?

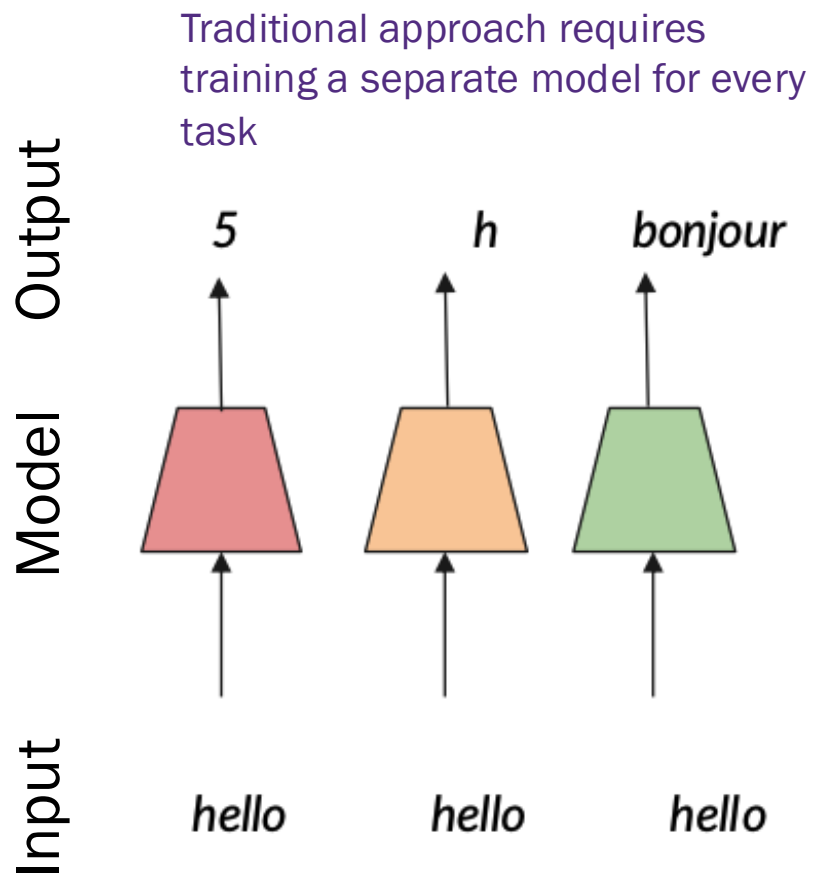


Machine learning paradigms

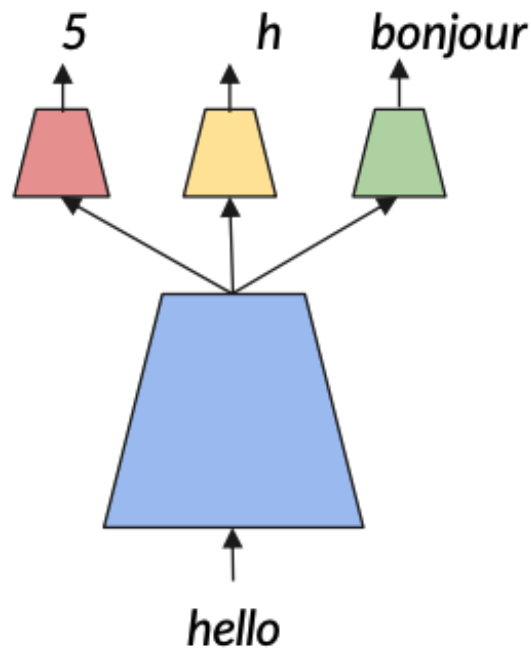




Machine learning paradigms

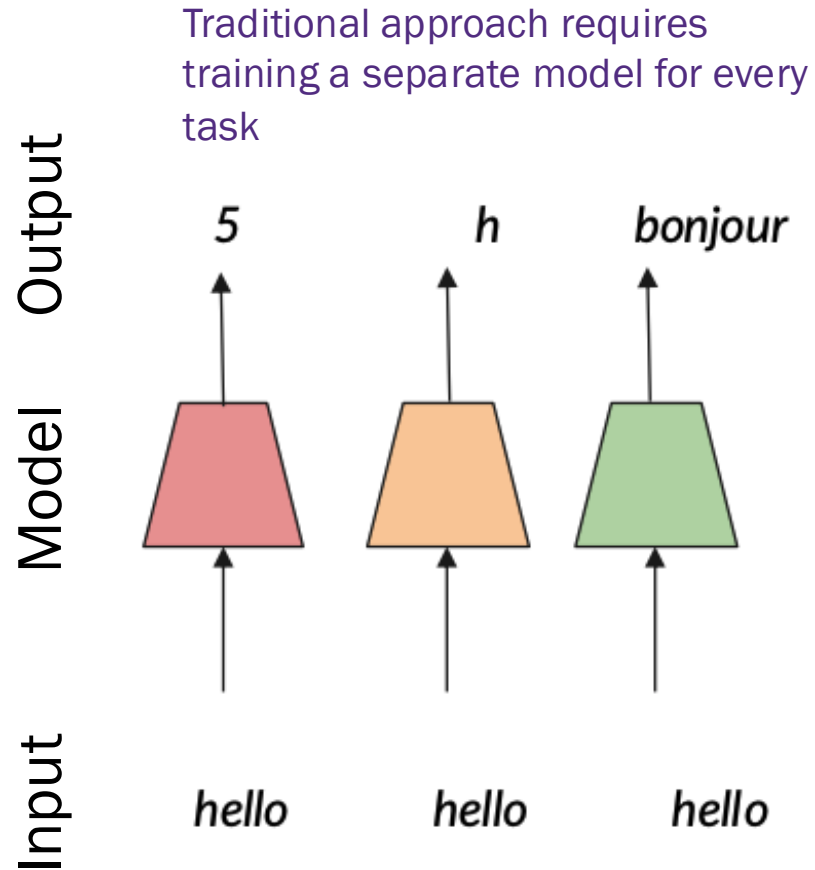


Finetuning (aka transfer learning)
"pre-trains" a model whose outputs are used in multiple downstream models that are individually trained. The pre-trained model may (or may not) be update during fine tuning.

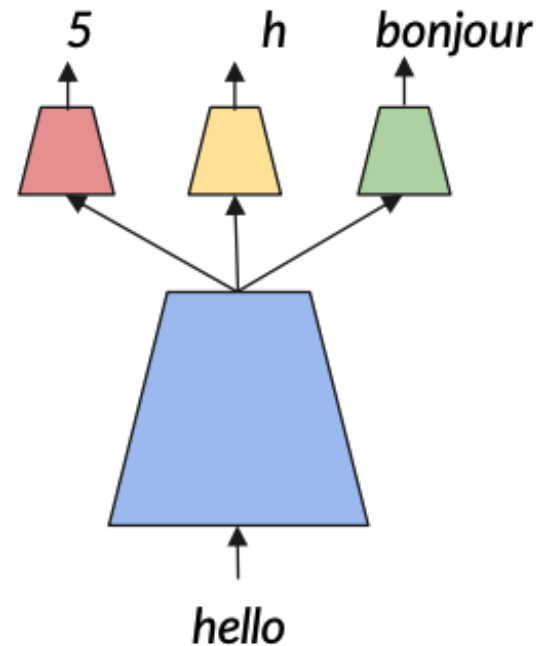




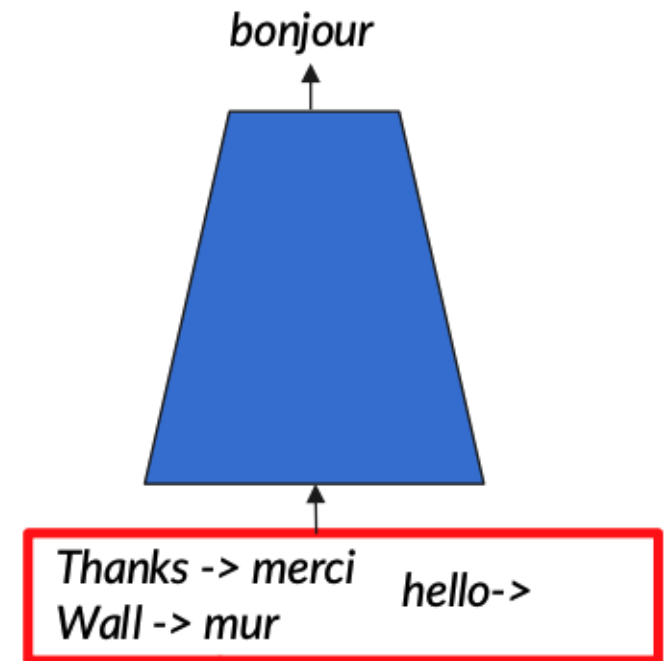
Machine learning paradigms



Finetuning (aka transfer learning) "pre-trains" a model whose outputs are used in multiple downstream models that are individually trained. The pre-trained model may (or may not) be update during fine tuning.



Few-shot prompting provides examples (context) to a model (e.g., LLM) for a task on which it was not explicitly trained and asks for the output on a new input.





In-context learning with LLMs

Modern LLMs can perform in context learning given natural language prompts that demonstrate the problem to be solved

AM

gaot => goat
sakne => snake
brid => bird
fsih => fish
readb => bread
tilhg => light
cmihp =>

Deciphering letter rearrang

cmihp => chimp

AM

thanks => merci
hello => bonjour
mint => menthe
wall => mur
otter => loutre
bread =>

Thinking about completing E

bread => pain

AM

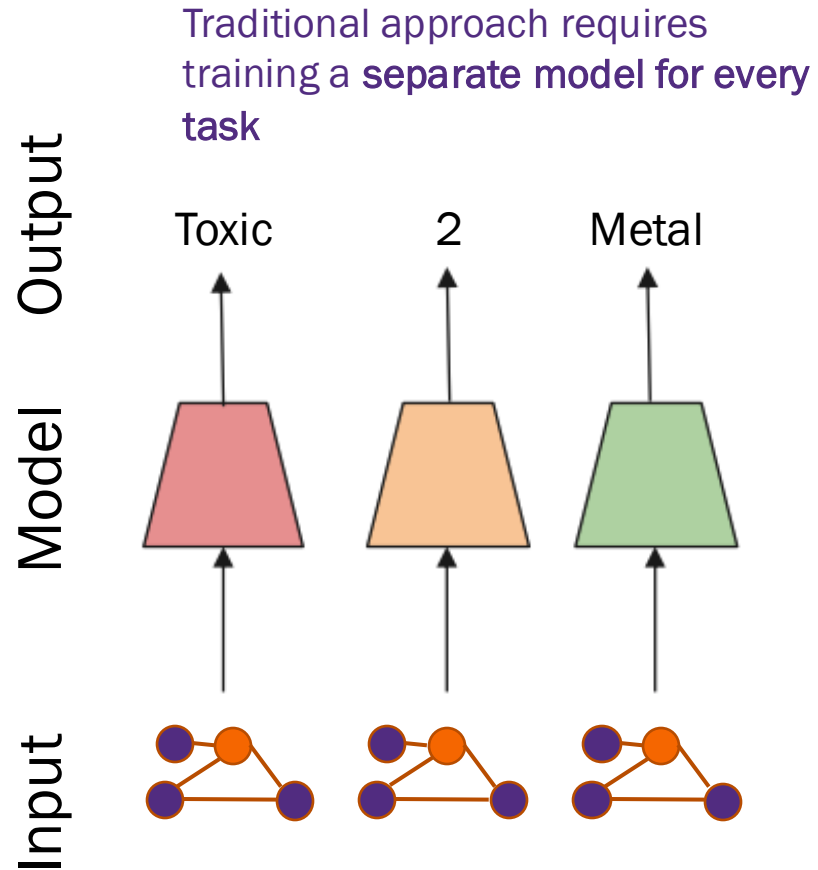
5, 8 => 40
7, 2 => 14
-7, 2 => -14
3, 4 => 12
4, 3 => 12
9, 8 => 72
10, 11 =>

Thinking about identify

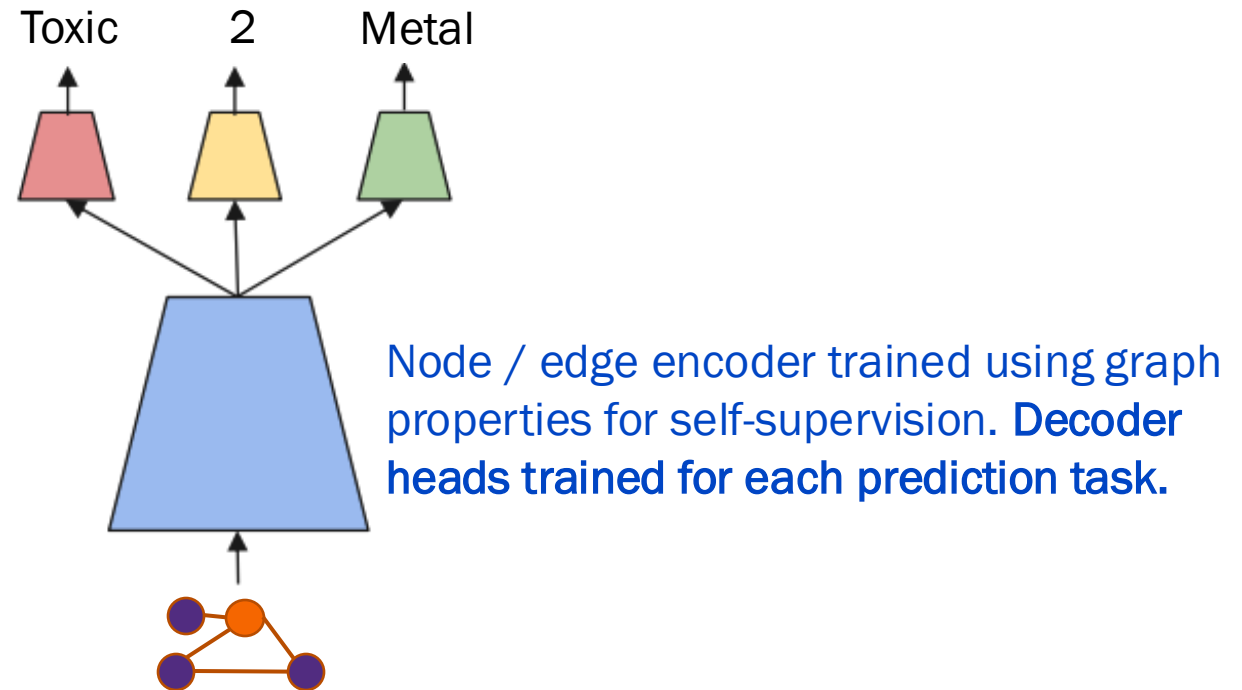
10, 11 => 110



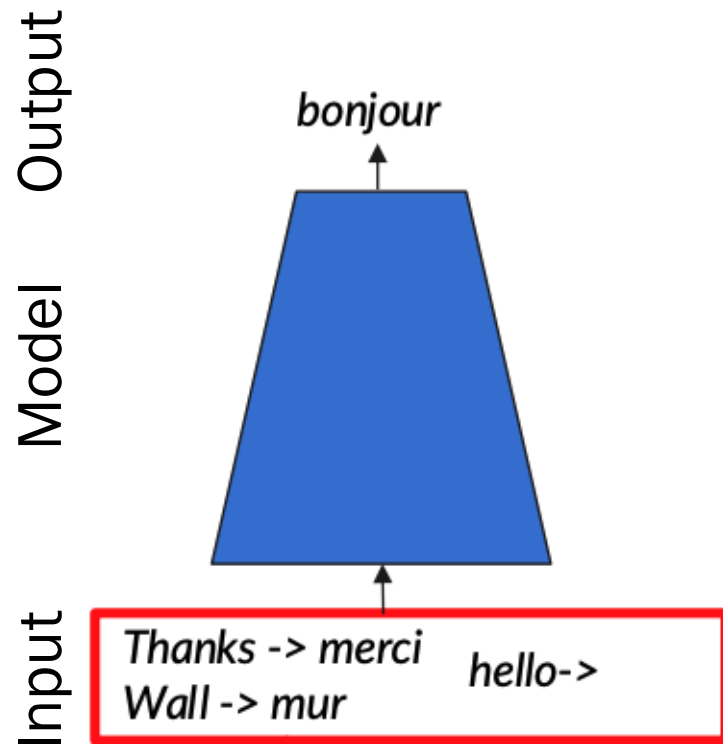
Machine learning paradigms on graphs



Finetuning (aka transfer learning)
"pre-trains" a model whose outputs are used in multiple downstream models that are individually trained. The pre-trained model may (or may not) be update during fine tuning.



In-context learning over graphs?



How can we formulate and implement in-context learning over graphs?



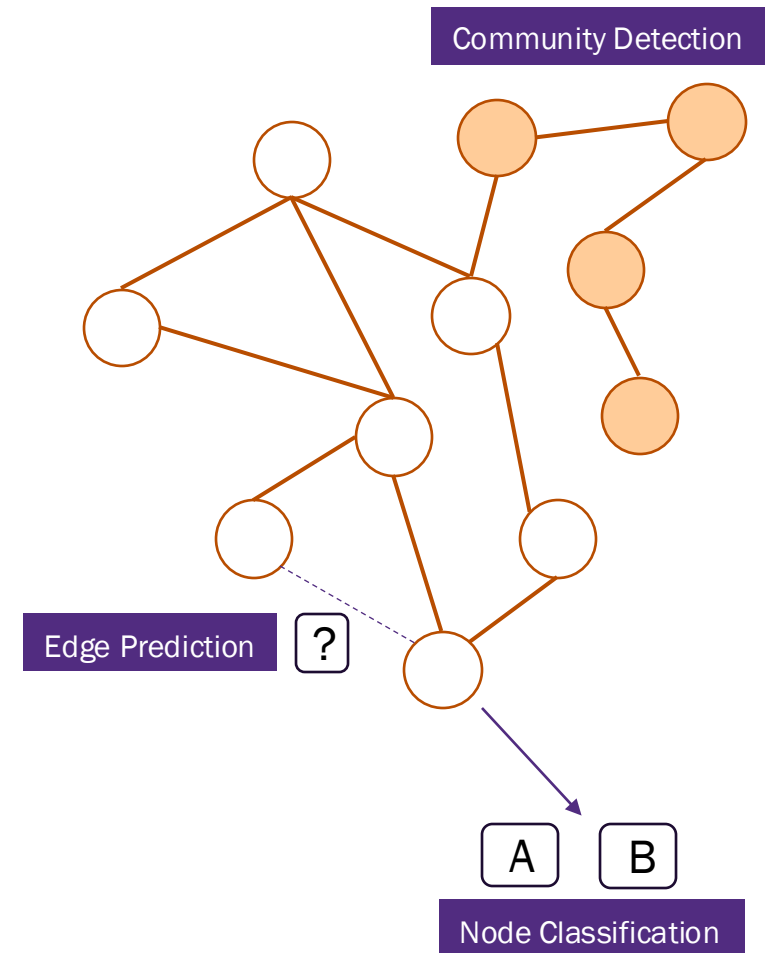
Formulation





Graph learning tasks: node, edge inference

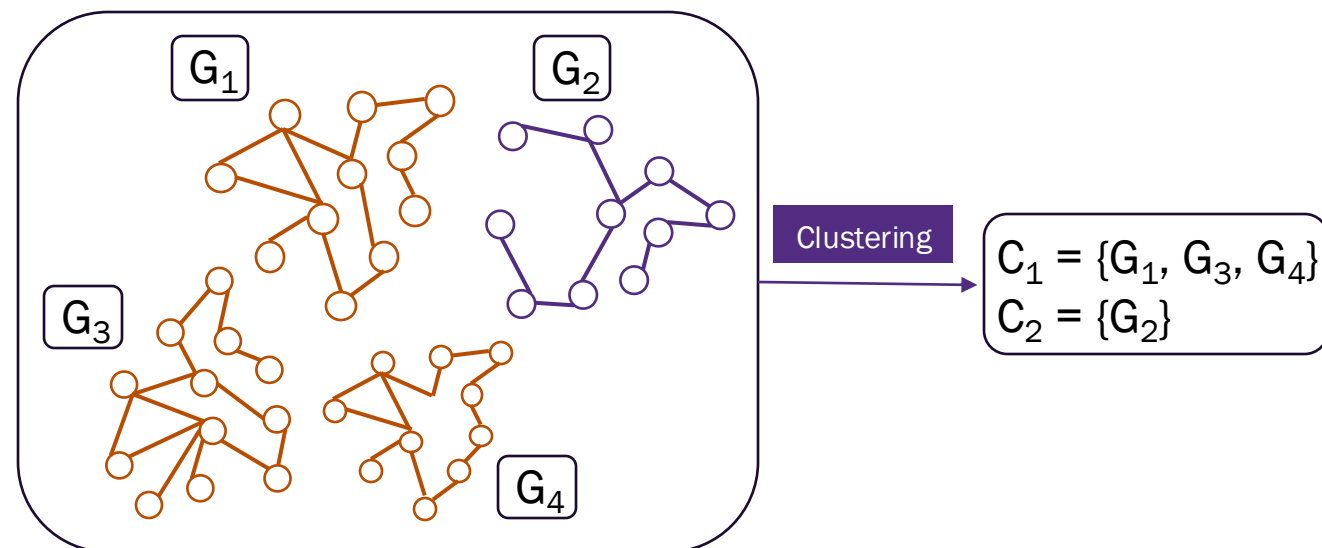
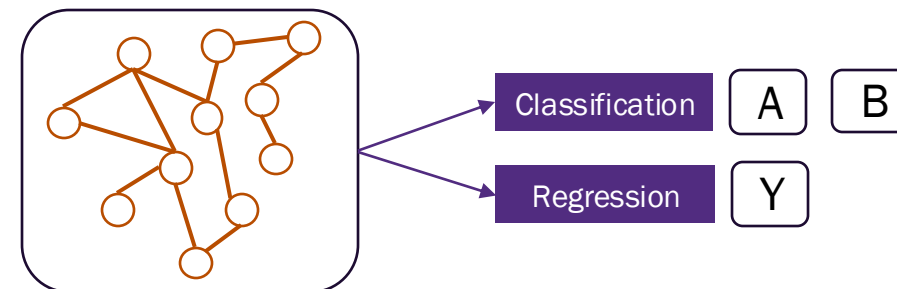
- Node classification – given information about the node (may include neighbors, relationships, attributes), infer class membership (usually an attribute)
- Edge prediction – given information about two nodes, determine if a given relation exists between them
- Community detection (node clustering) – detection of “meaningful” node groups





Graph learning tasks: graph regression / classification

- Whole graph classification and regression – given one entire graph, infer class membership or estimate (regress) a related numerical property
- Inter graph tasks – given multiple graphs estimate the similarity between graphs and/or identify clusters

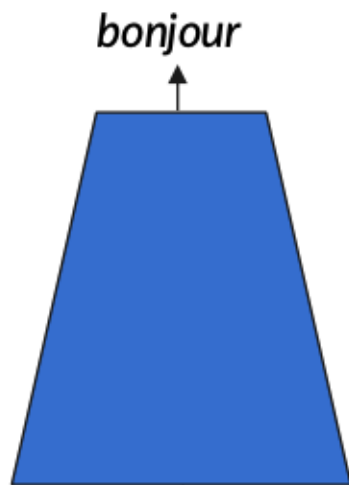




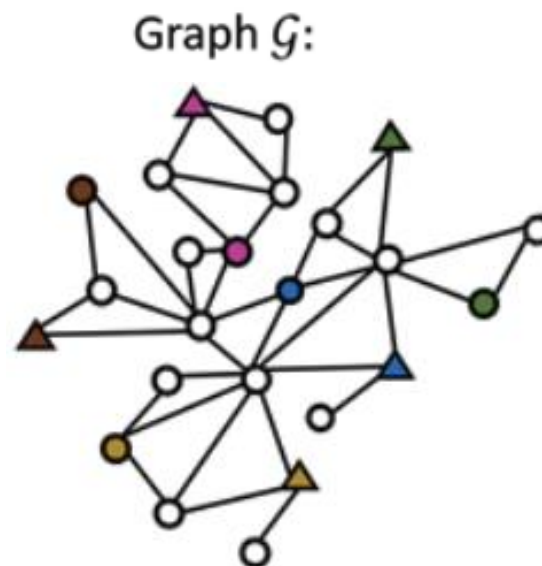
In-context learning over graphs?

Input
Model
Output

Thanks -> merci
Wall -> mur



bonjour



Different graphs

An in-context learner for graphs should be able to solve new tasks **without learning parameter updates!**

Prompt Examples $\mathcal{S}$:

Input x	Label y
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacksquare$
$(\bullet, \blacktriangle)$	$\blacksquare$

Queries Q :

$(\bullet, \blacktriangle) \rightarrow \blacklozenge \text{ OR } \blacksquare ?$

Different tasks



Challenges for in-context learning over graphs

1. How can the few-shot prompt for **different graph tasks** be represented in a **single format** to serve as model input?
2. How can a model be **pre-trained** to produce this emergent capability?



Challenges for in-context learning over graphs

1. How can the few-shot prompt for **different graph tasks** be represented in a **single format** to serve as model input?
 - **Prompt Graph**: represent each prompt example and query (task) with a meta hierarchical graph.
2. How can a model be **pre-trained** to produce this emergent capability?
 - **PGPretraining**: Pretrain a message passing model over self-supervised tasks in PromptGraph format with diverse underlying structures



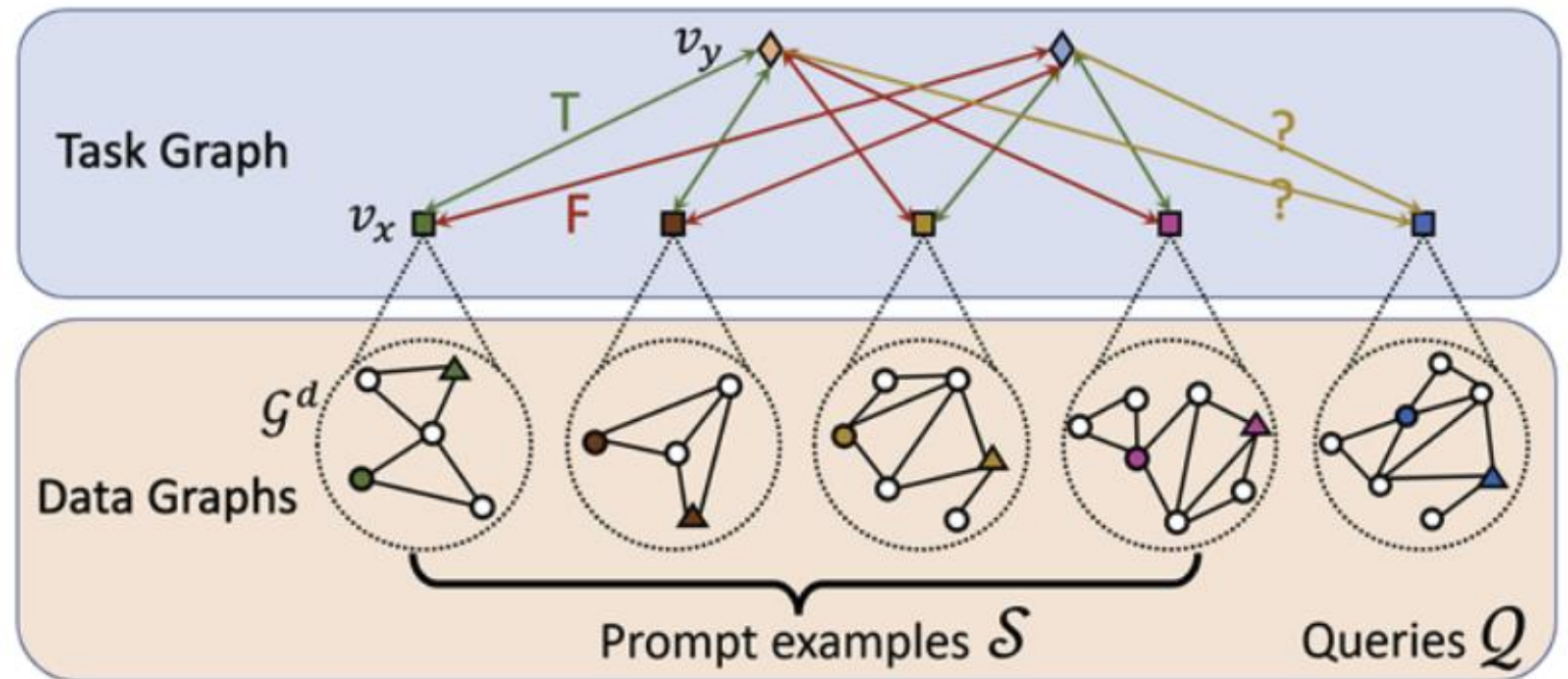
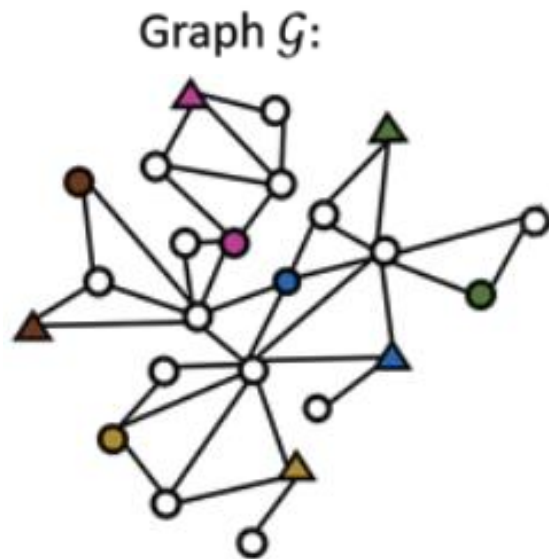
Prompt Graph





Prompt Graph overview

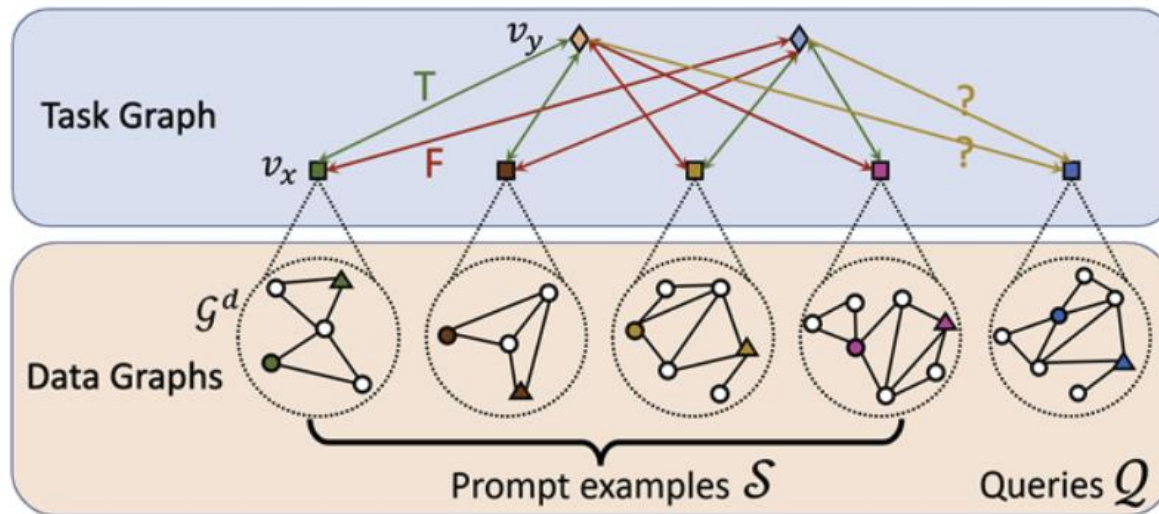
- Consider a source graph, $\mathcal{G}$, and a task (e.g. node classification)
- Prompt Graph is a unified representation of examples and queries





Prompt Graph overview

- Prompt graph has two graphs
 - Data graphs
 - Task graph



Data Graphs (collection of graphs)

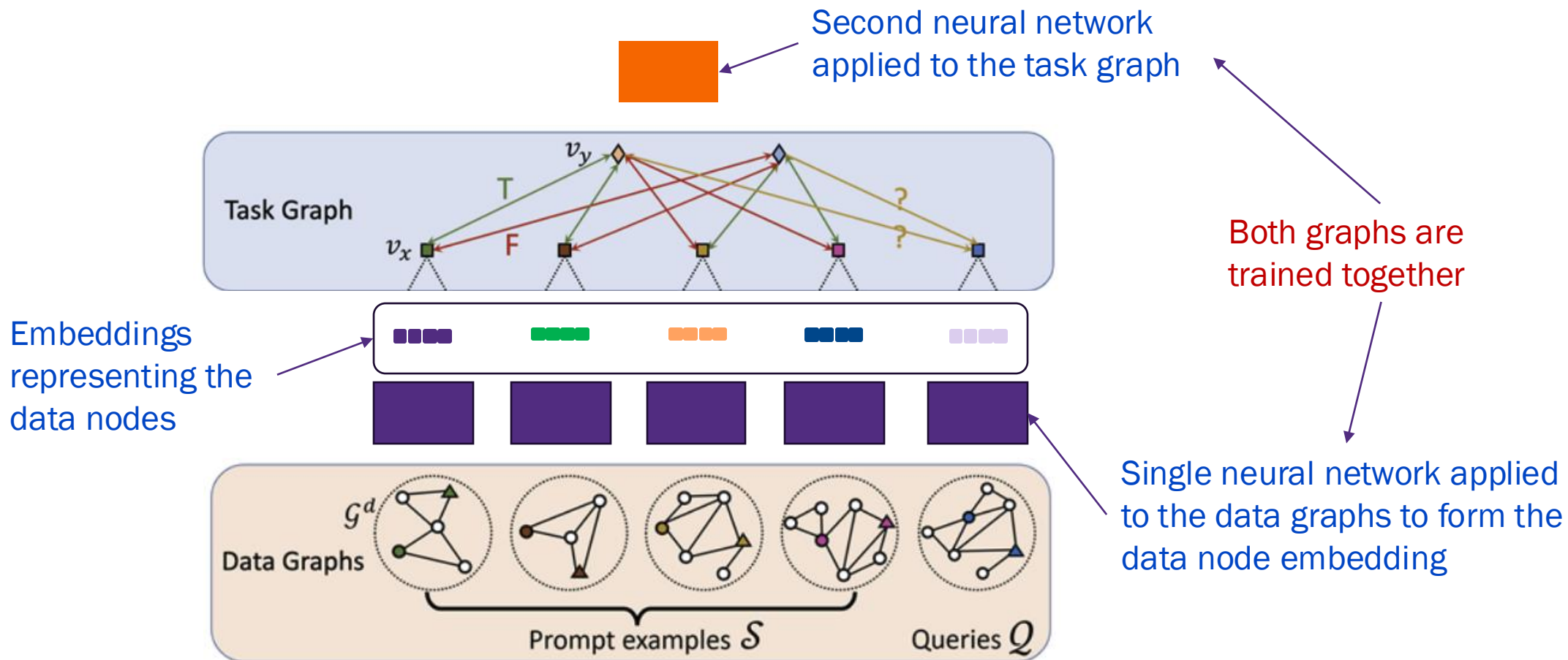
- Examples (shots) that include local graph structure and labels that illustrate the task
- Queries that include the local graph structure on which model inference is required

Task Graph

- Includes a single *data node* for each data graph
- Includes all possible outcome labels as nodes
- Includes an edge between each data node and each label
 - T / F feature indicating if the label applies to the data node
 - Example / query feature indicating if the data node is from an example or a query

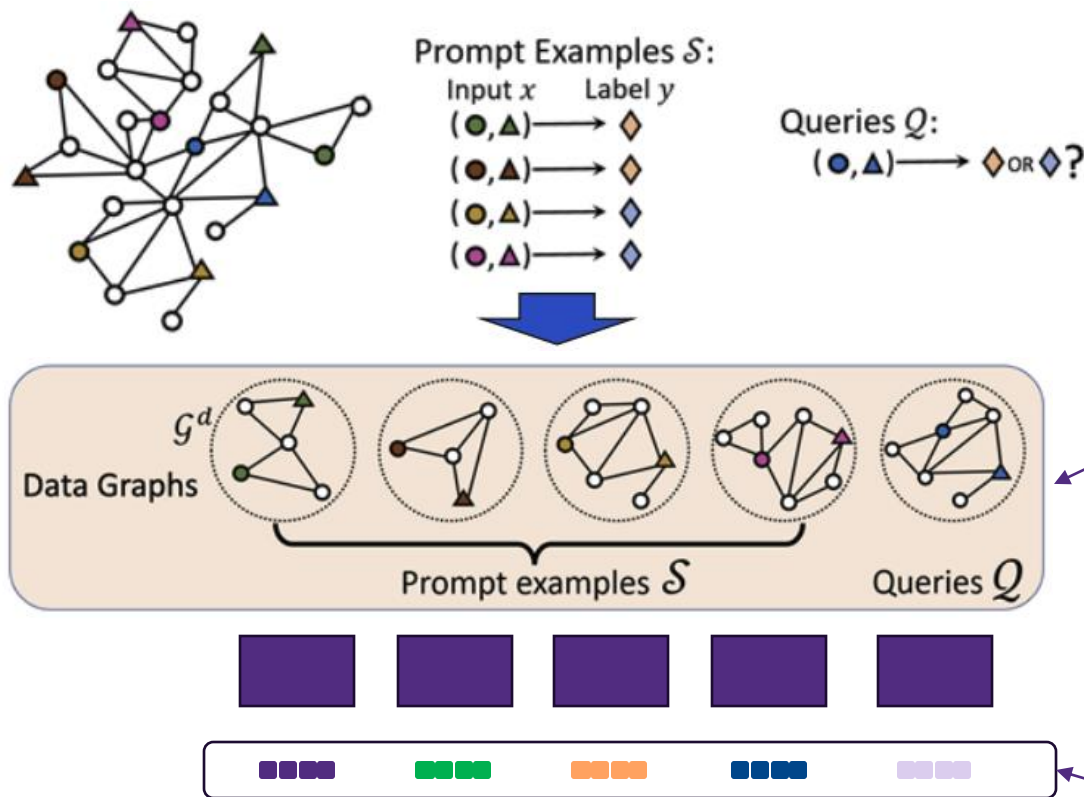


Connecting the data and task graphs



Data graph for link prediction

Data graph contextualizes the input (a pair of nodes) in the graph $\mathcal{G}$ (e.g., by subgraph extraction)



Each data graph, $\mathcal{G}_i^d$, (examples and queries) has a pair of nodes, (u_i, v_i) , for which link prediction is required. The models do not "know" the pair, (u_i, v_i) .

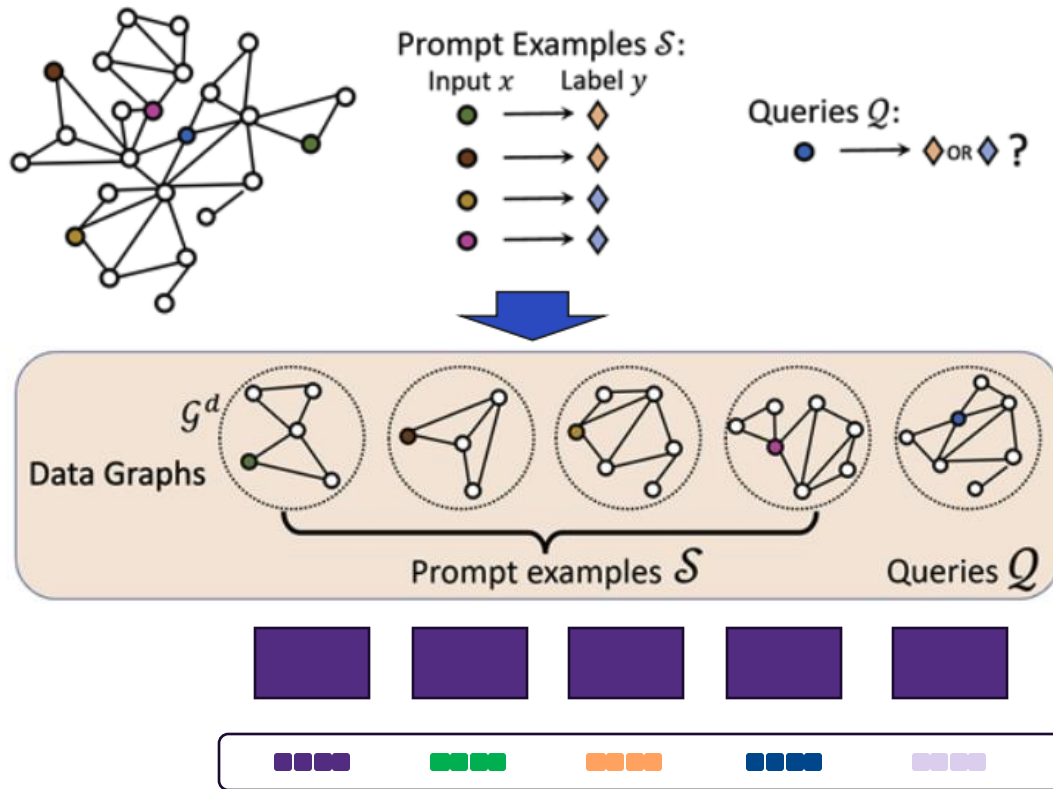
For link prediction, the *data node* in the task graph is represented with the embedding given by:

$$\mathbf{W}^T[\mathbf{u}_i \parallel \mathbf{v}_i \parallel \mathbf{e}_i] + b$$

where $\mathbf{W}$ is a learnable matrix, $\mathbf{e}_i$ is the element wise max over all node embeddings in $\mathcal{G}_i^d$, and b is a bias term

Data graph for node classification

Data graph contextualizes the input (a node) in the graph $\mathcal{G}$ (e.g., by subgraph extraction)



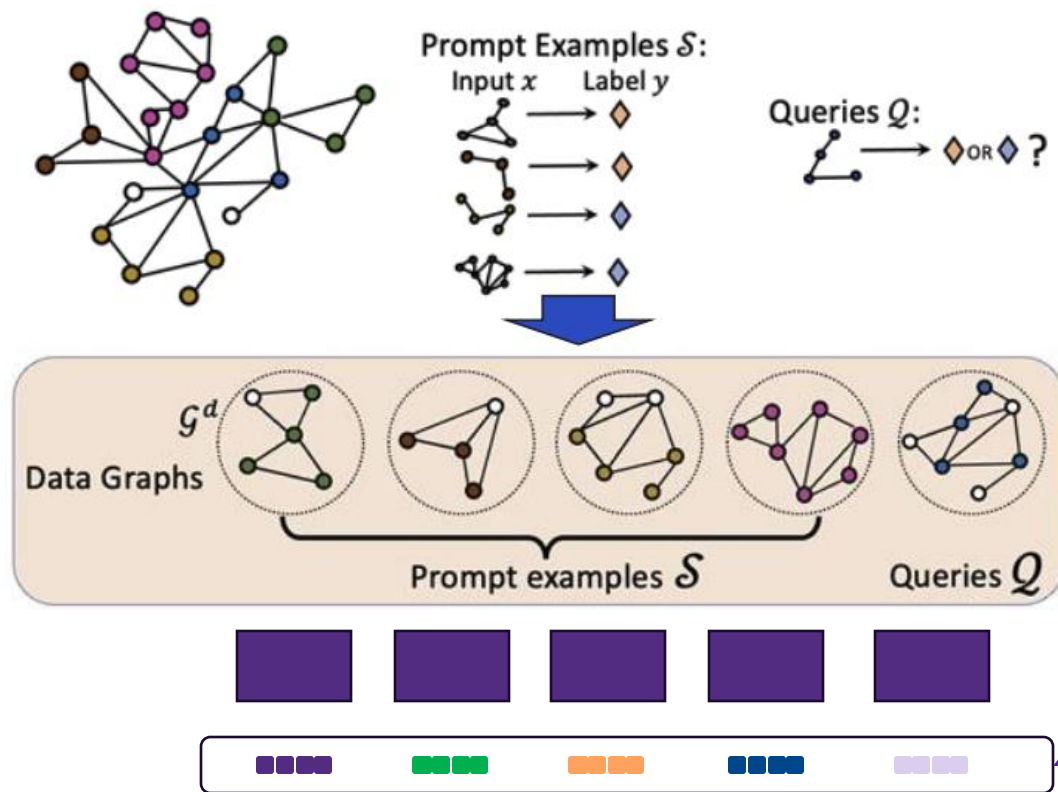
Each data graph, $\mathcal{G}_i^d$, (one for each example and query) has a specific node, v_i , that is to be classified. The models do not "know" which node is v_i .

For node classification, the embedding, $\mathbf{v}_i$ of the node, v_i , is used to represent the *data node* for data graph, $\mathcal{G}_i^d$, in the task graph.



Data graph for (sub)graph classification

Data graph contextualizes the input (a (sub)graph) in the graph $\mathcal{G}$ (e.g., by subgraph extraction)

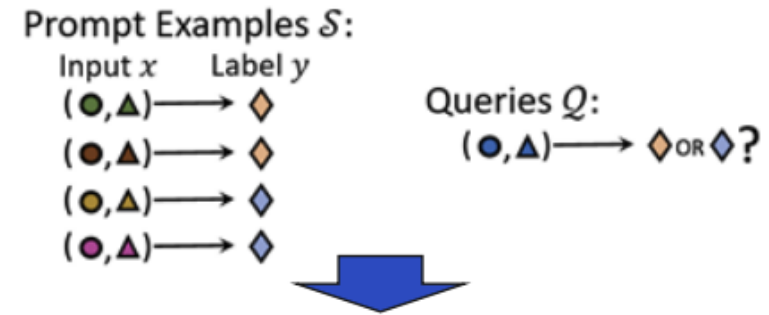


Each data graph, $\mathcal{G}_i^d$, (examples and queries) is a **graph or subgraph** with nodes V_i , for which classification is required. The models do not "know" the set, V_i .

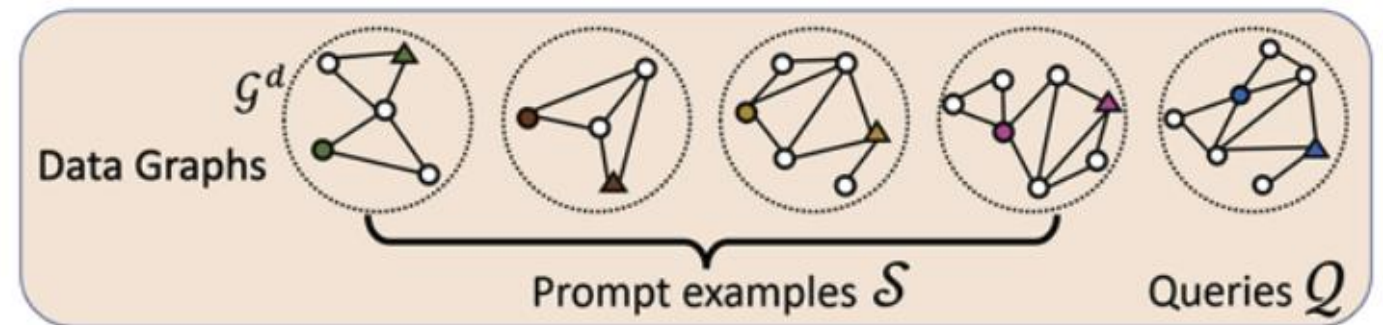
For (sub)graph prediction, the *data node* in the task graph is represented with the embedding given by element wise global pooling (e.g., mean, max) over all nodes in V_i .

Task graph for link prediction

Task graph interconnects inputs and labels across examples to form context for queries



How do we construct the task graph given the input samples and queries?



Task graph for link prediction

Task graph interconnects inputs and labels across examples to form context for queries

Prompt Examples $\mathcal{S}$:

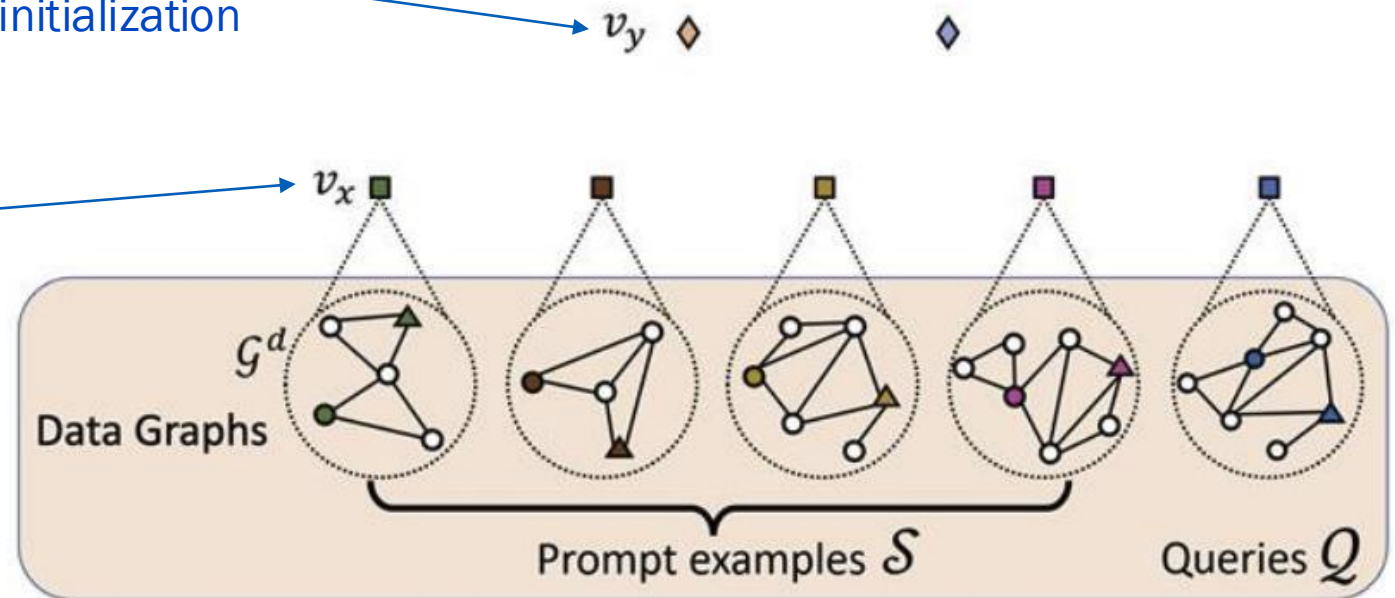
Input x	Label y
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacklozenge$

Queries $\mathcal{Q}$:

$(\bullet, \blacktriangle) \rightarrow \blacklozenge \text{ OR } \blacklozenge ?$

Label nodes with embeddings that have random Gaussian initialization

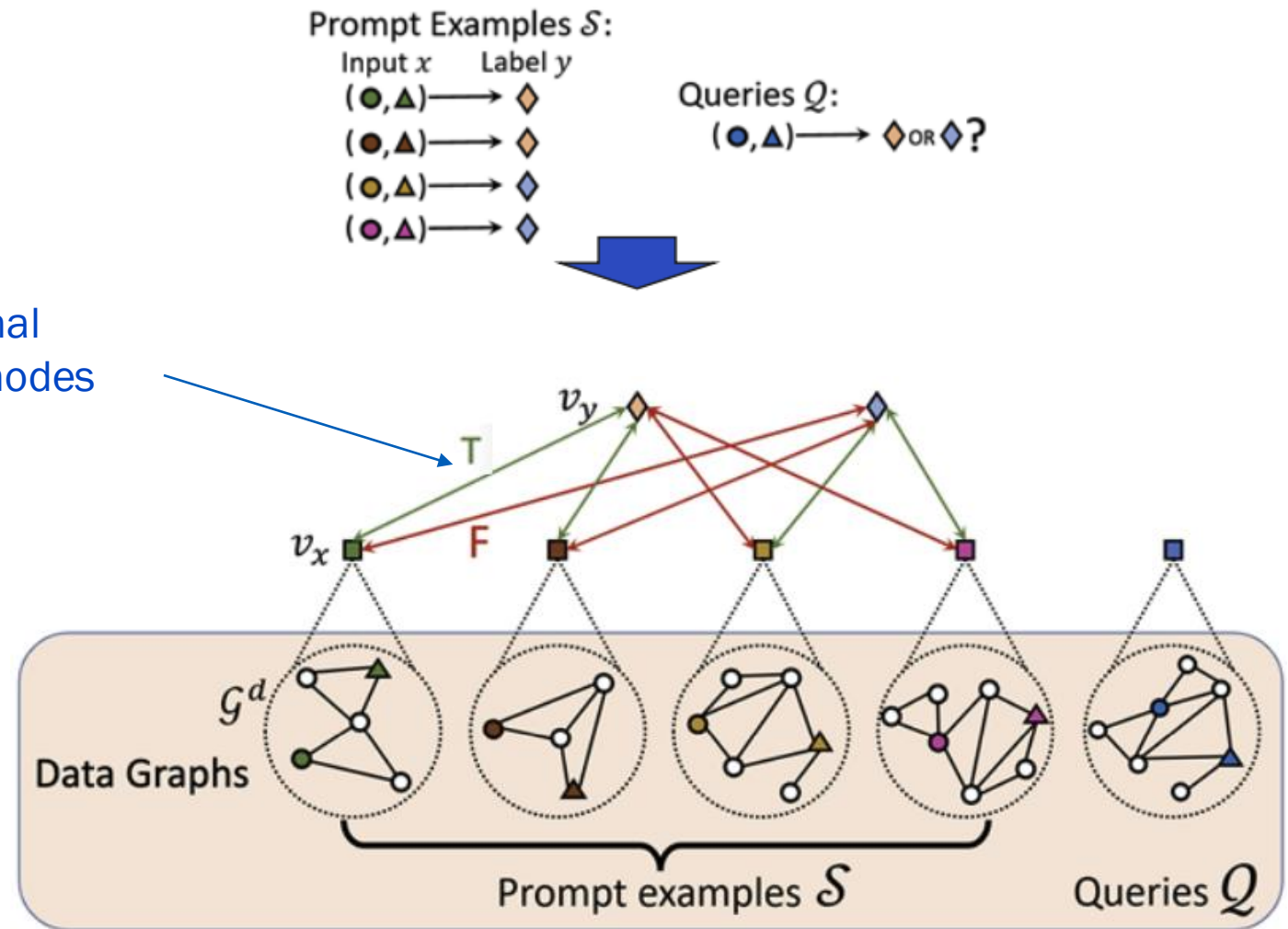
Data nodes with embeddings generated from the network applied to each data graph



Task graph for link prediction

Task graph interconnects inputs and labels across examples to form context for queries

Prompt examples: Bidirectional edges added between data nodes and label nodes.



Task graph for link prediction

Task graph interconnects inputs and labels across examples to form context for queries

Prompt examples: Bidirectional edges added between data nodes and label nodes.

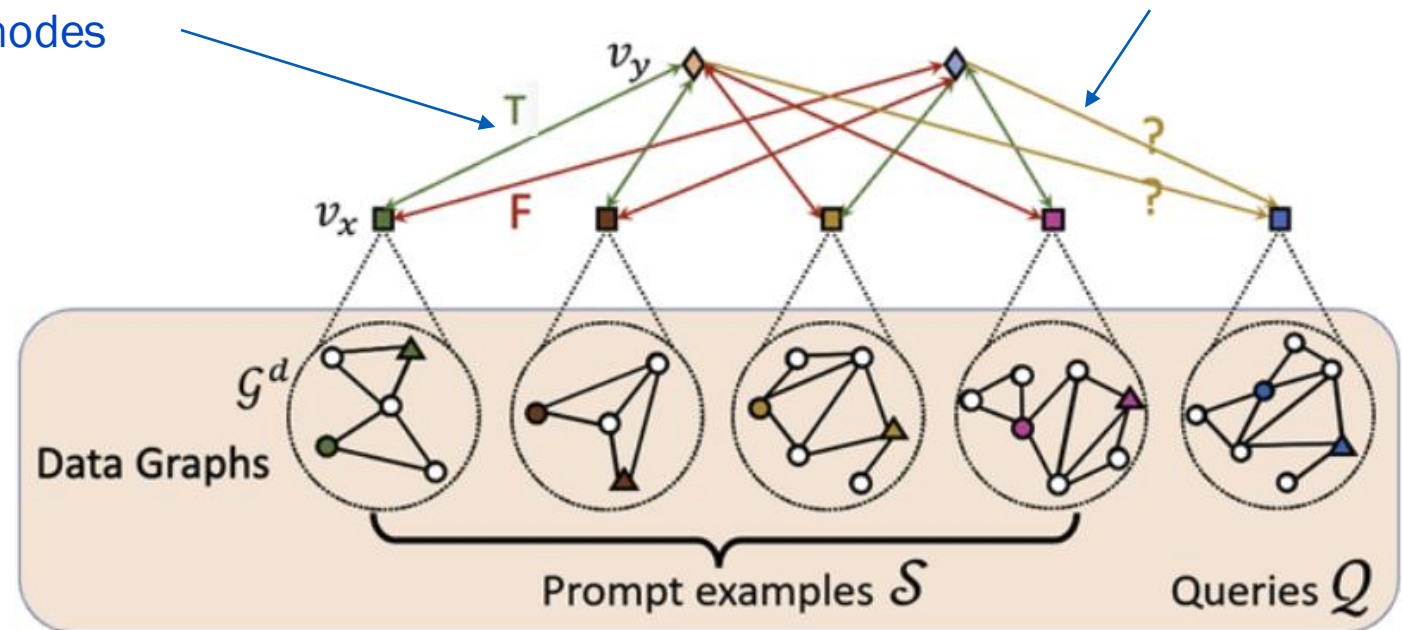
Prompt Examples $\mathcal{S}$:

Input x	Label y
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacklozenge$

Queries $\mathcal{Q}$:

$(\bullet, \blacktriangle) \rightarrow \blacklozenge \text{ OR } \blacklozenge ?$

Queries: Directed edge from each label node to each data node



Task graph for link prediction

Task graph interconnects inputs and labels across examples to form context for queries

Prompt examples: Bidirectional edges added between data nodes and label nodes.

Edge features

- T / F feature indicating if the label applies to the data node (set to F for all query nodes)
- Example / query feature indicating if the data node is from an example or a query

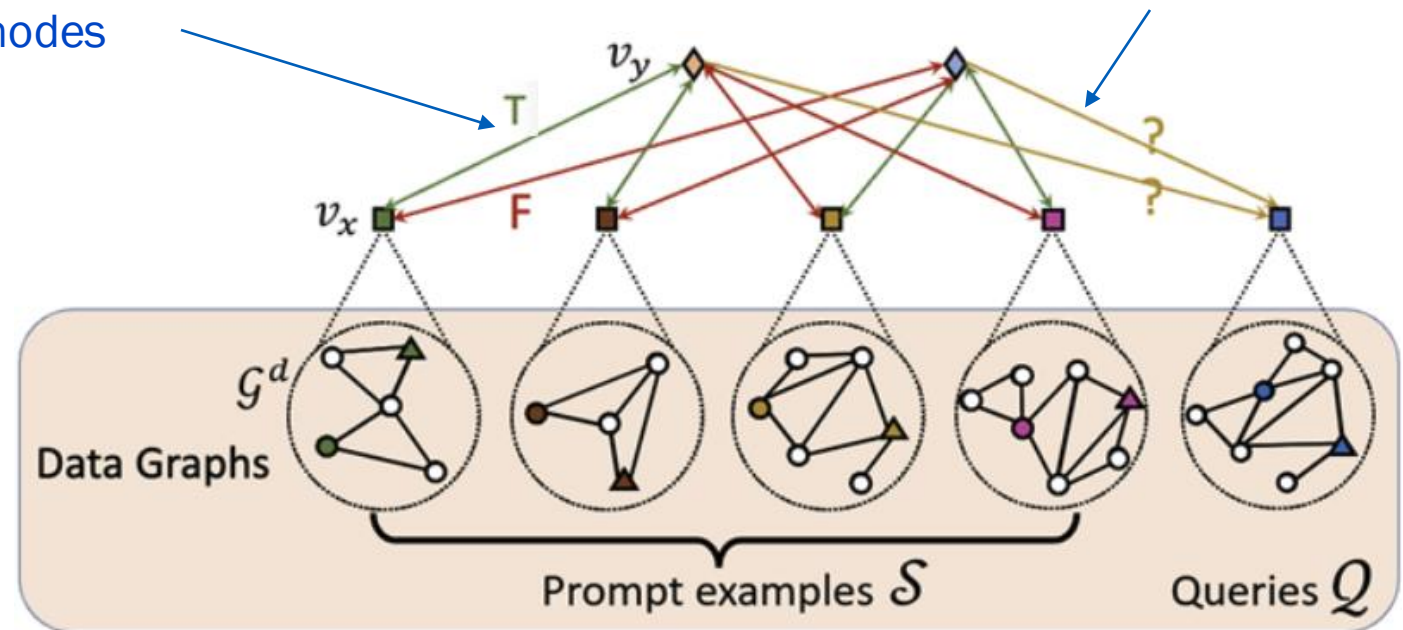
Prompt Examples $\mathcal{S}$:

Input x	Label y
$(\bullet, \blacktriangle) \rightarrow$	$\blacklozenge$
$(\bullet, \blacktriangle) \rightarrow$	$\blacklozenge$
$(\bullet, \blacktriangle) \rightarrow$	$\blacklozenge$
$(\bullet, \blacktriangle) \rightarrow$	$\blacklozenge$

Queries $\mathcal{Q}$:

$(\bullet, \blacktriangle) \rightarrow \blacklozenge \text{ OR } \blacklozenge ?$

Queries: Directed edge from each label node to each data node



Task graph for link prediction

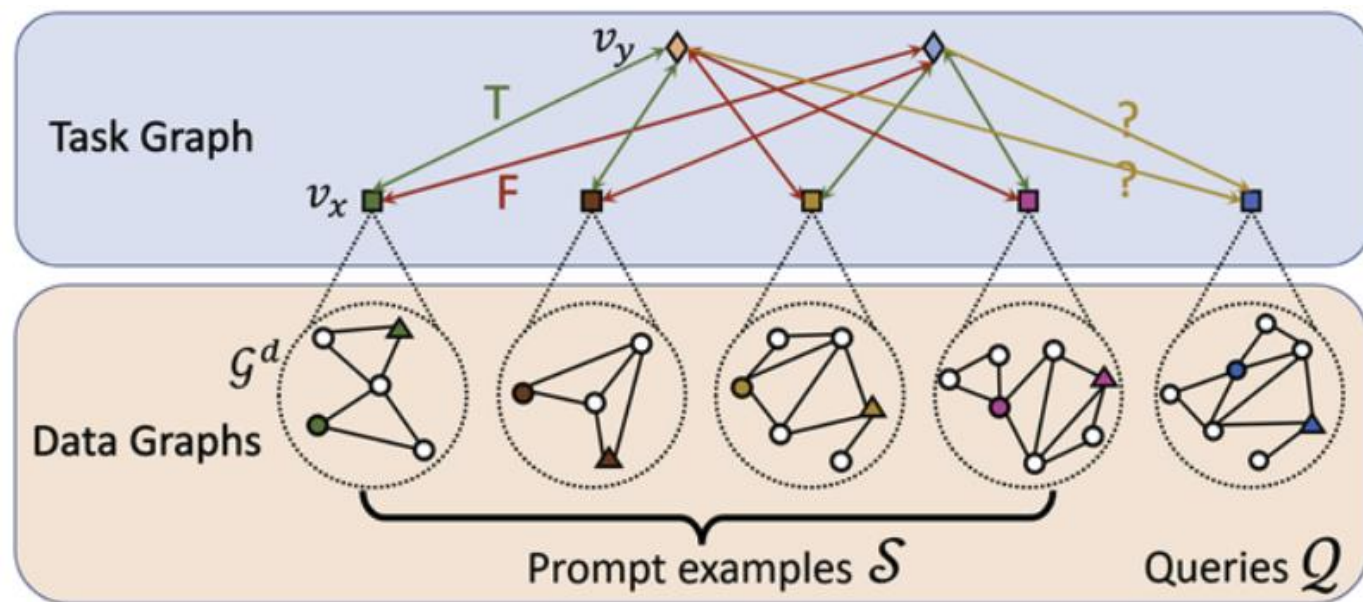
Task graph interconnects inputs and labels across examples to form context for queries

Prompt Examples $\mathcal{S}$:

Input x	Label y
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacklozenge$
$(\bullet, \blacktriangle)$	$\blacklozenge$

Queries $\mathcal{Q}$:

$(\bullet, \blacktriangle) \rightarrow \blacklozenge \text{ OR } \blacklozenge ?$



Task graph for node classification

Task graph interconnects inputs and labels across examples to form context for queries

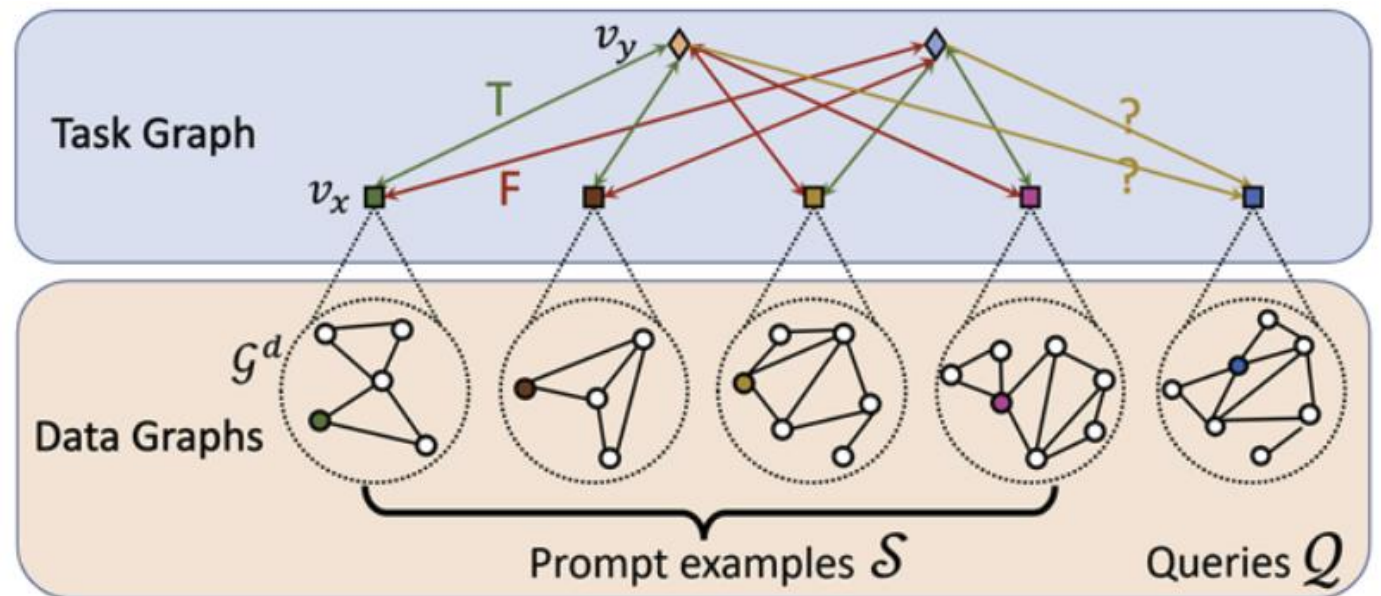
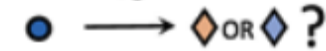
Same as link predictions except that the data nodes are node-level embeddings from the data graphs

Prompt Examples $\mathcal{S}$:

Input x Label y



Queries $\mathcal{Q}$:





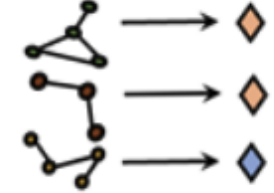
Task graph for (sub)graph classification

Task graph interconnects inputs and labels across examples to form context for queries

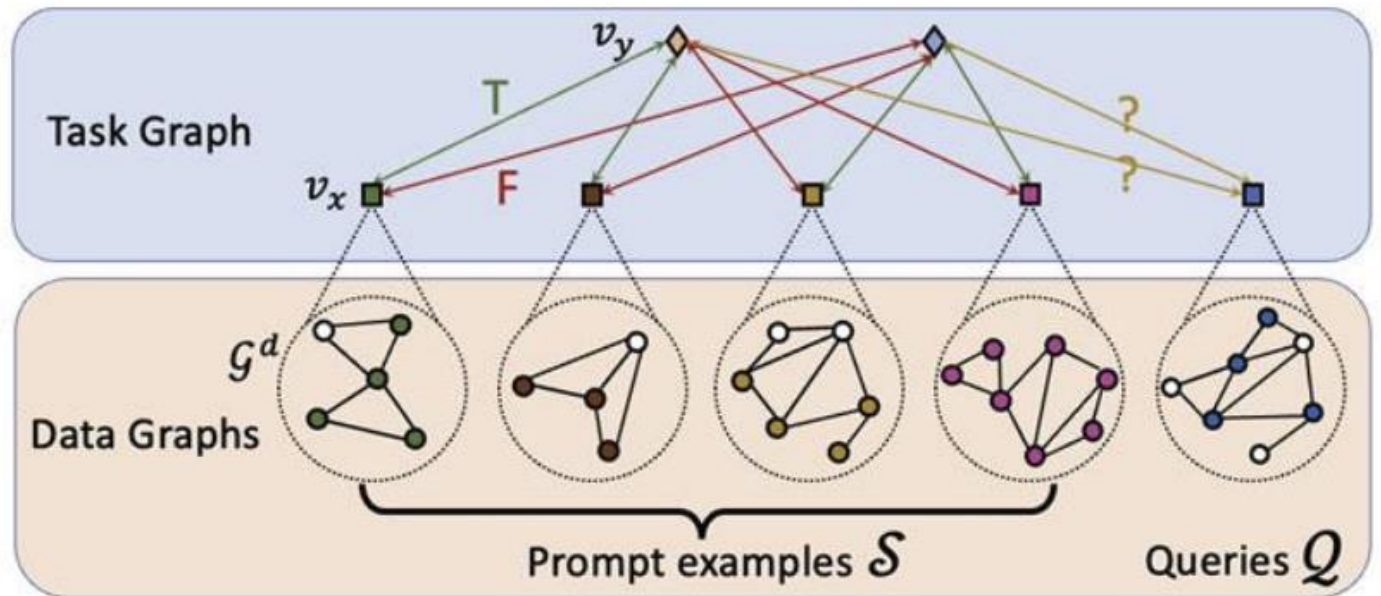
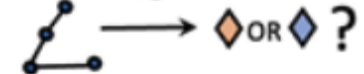
Same as link predictions except that the data nodes are graph-level embeddings from the data graphs

Prompt Examples $\mathcal{S}$:

Input x Label y



Queries Q :

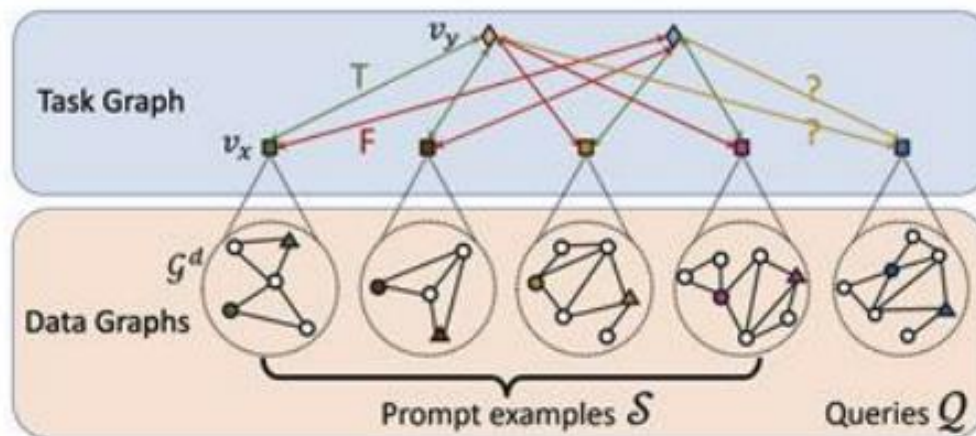




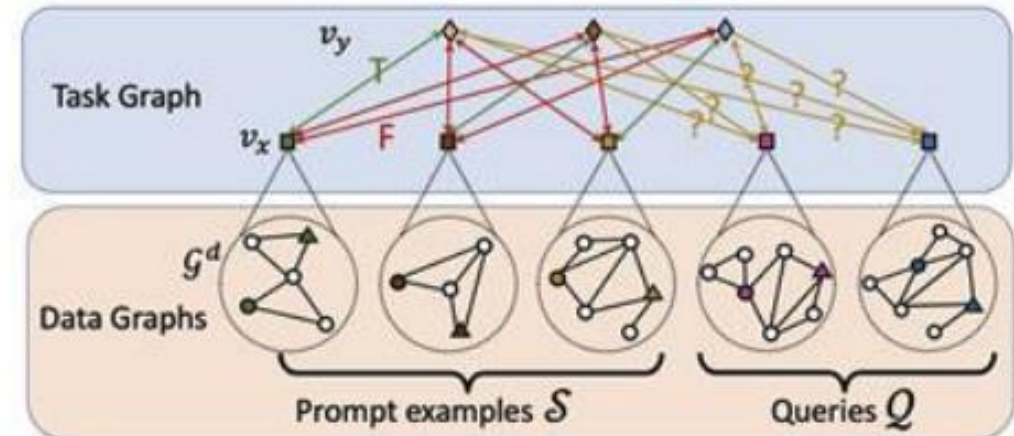
Task graph flexibility

Task graph unifies classification task formulation

- Different numbers of classes are represented as different number of label nodes
- Different number of prompt examples (shots) and queries are represented as different number of data nodes



2-shots prompt for 2-class classification (Each “shot” includes a sample for each class label)



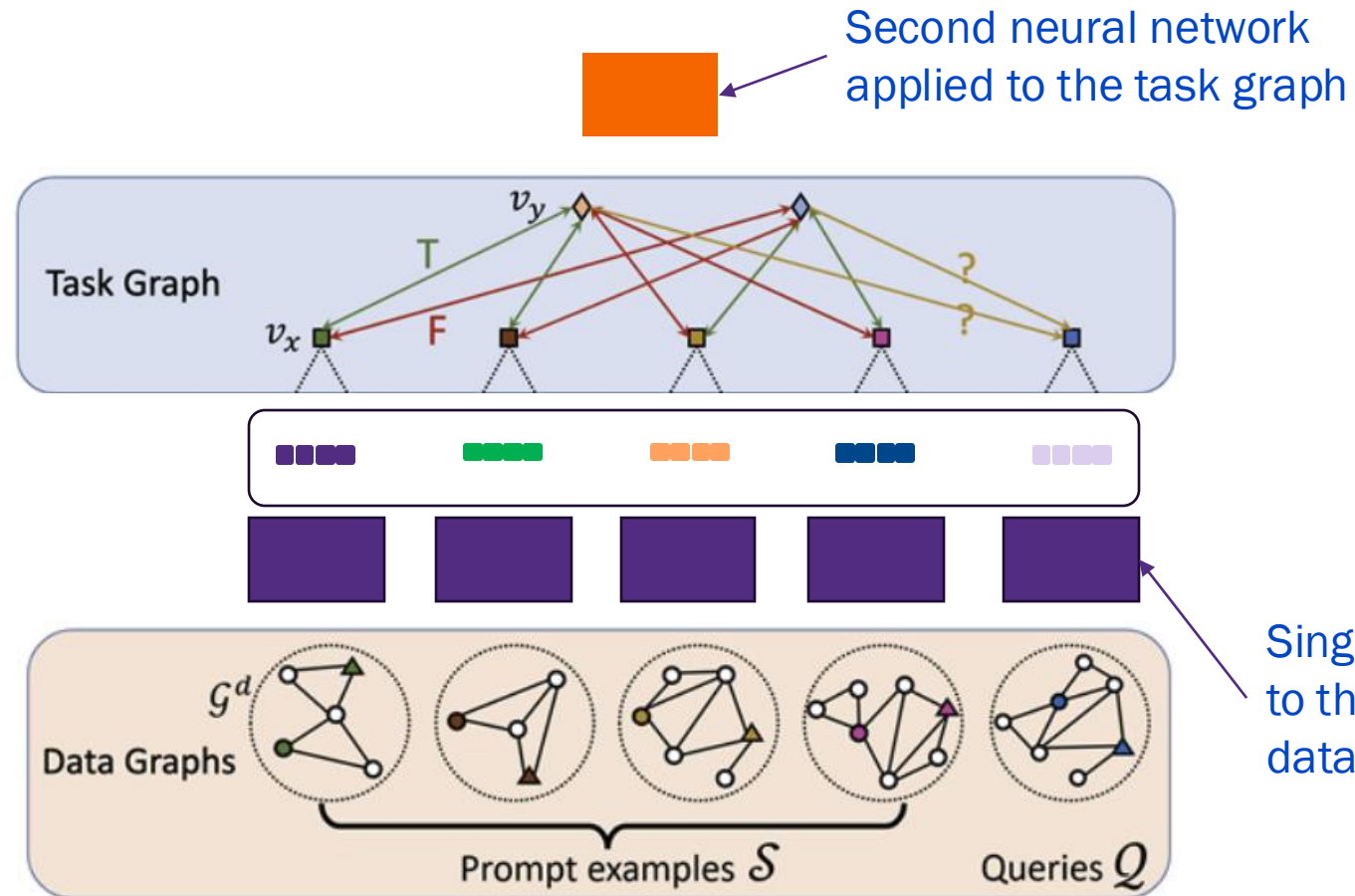
1-shot prompt for c-class classification (Each “shot” includes a sample for each class label)



Prompt graph inference



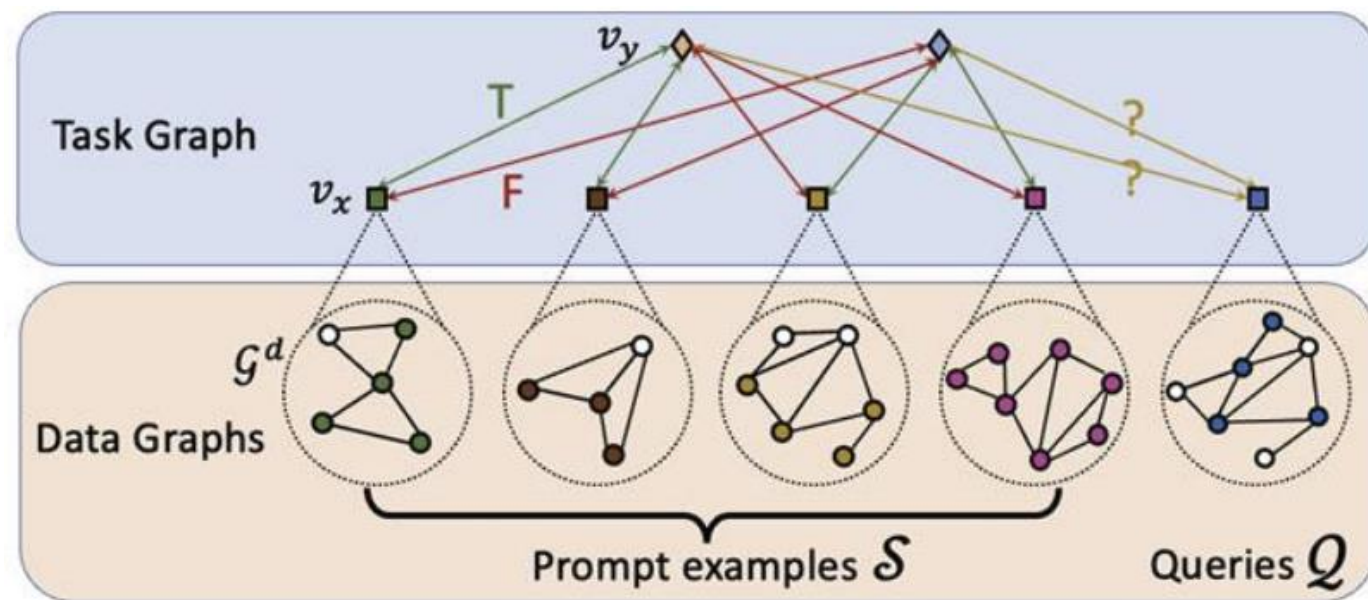
In-context inference (prediction)





In-context inference (prediction)

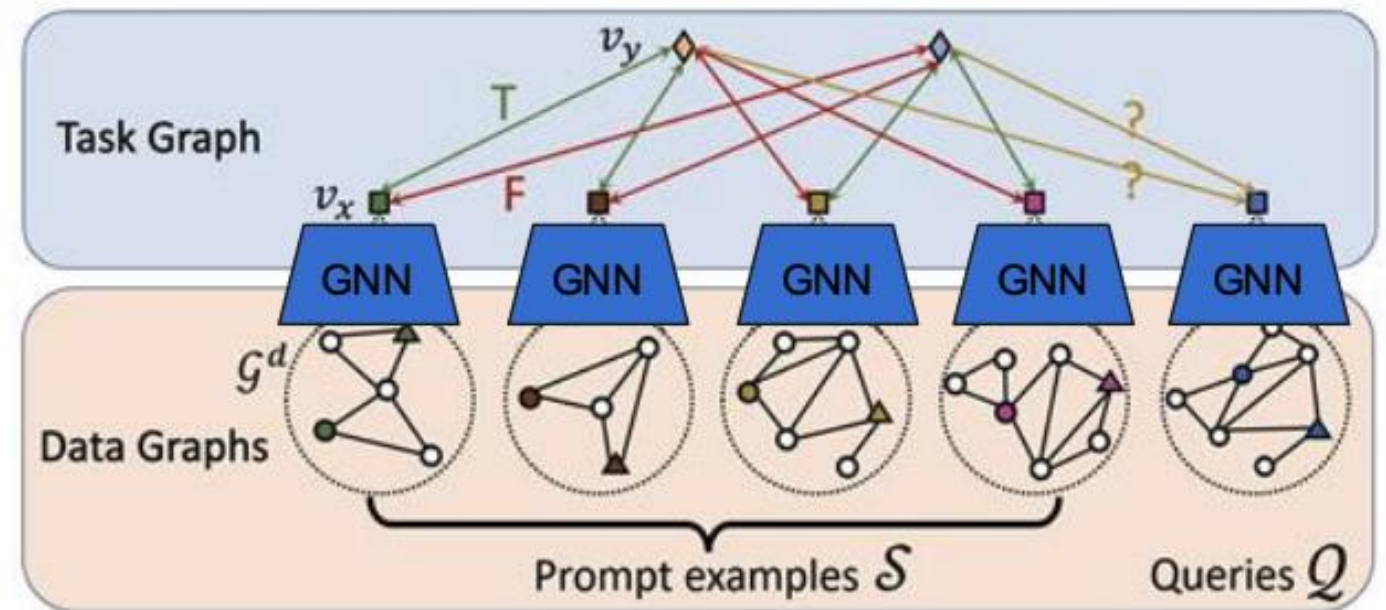
Perform *hierarchical* message passing over the Prompt Graph



In-context inference (prediction)

Perform *hierarchical* message passing over the Prompt Graph

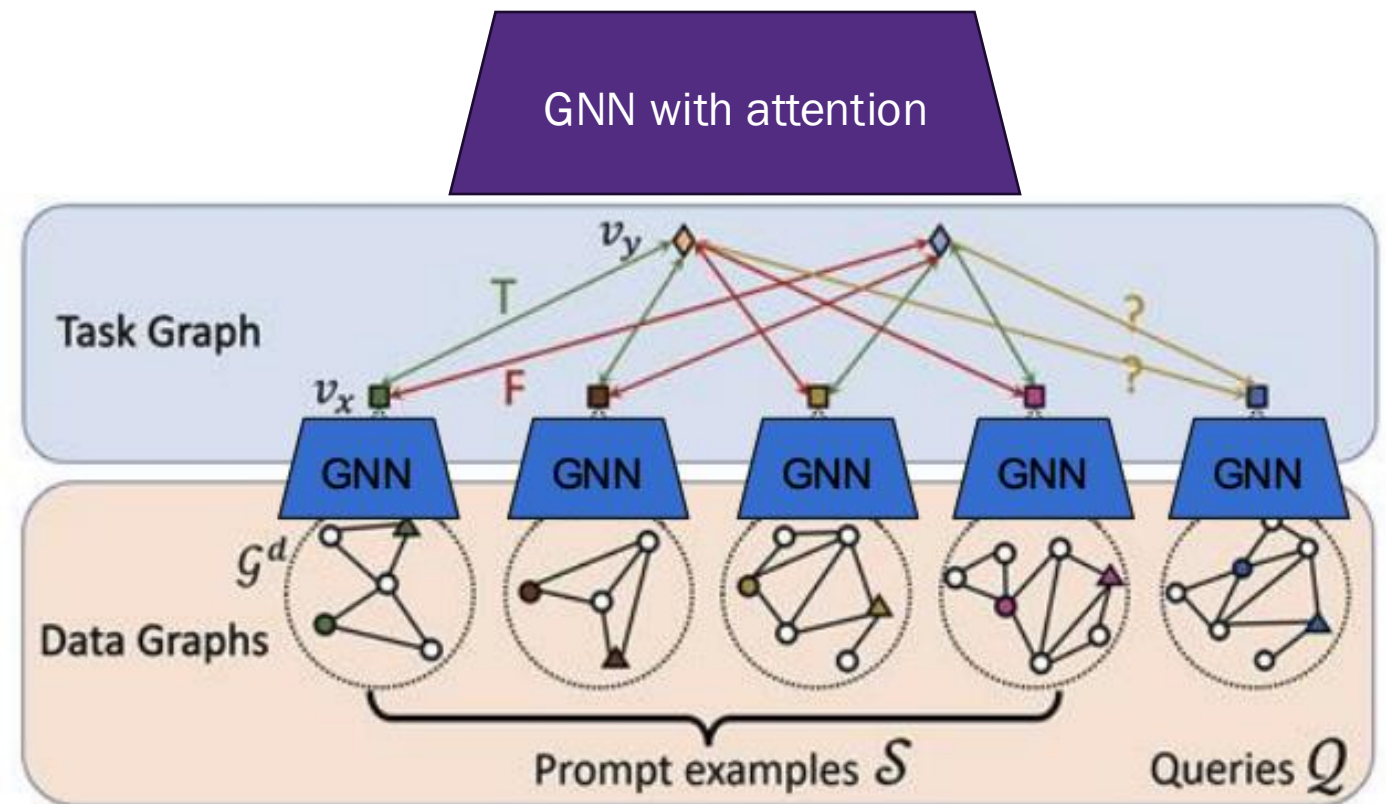
- GNN over data graphs (e.g., GCN, GAT) – same GNN processes each data graph separately to produce data node embeddings



In-context inference (prediction)

Perform *hierarchical* message passing over the Prompt Graph

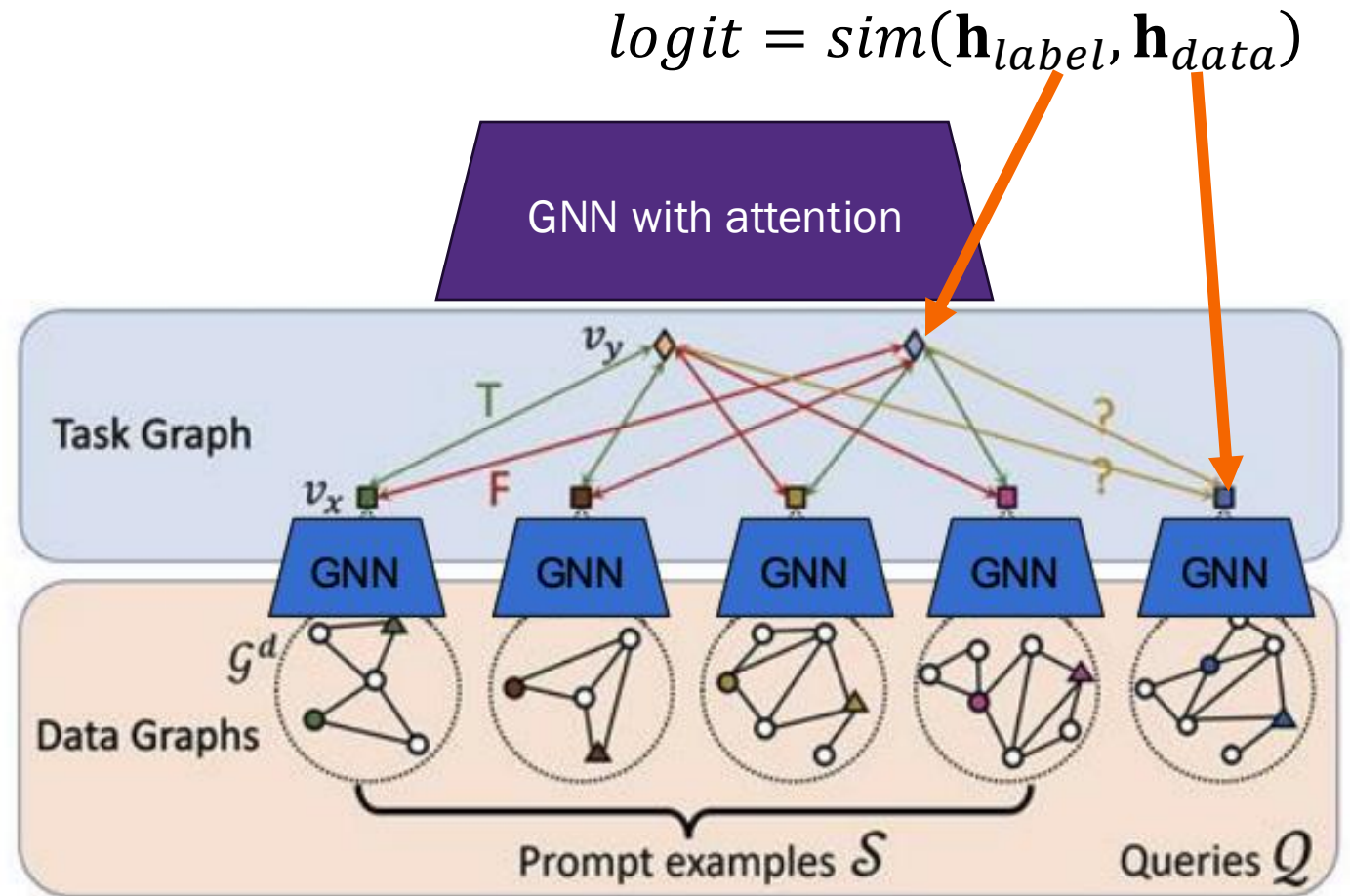
- GNN over data graphs (e.g., GCN, GAT) – same GNN processes each data graph separately to produce data node embeddings
- GNN with attention over task graph - produces updated embeddings for labels and data nodes



In-context inference (prediction)

Perform *hierarchical* message passing over the Prompt Graph

- GNN over data graphs (e.g., GCN, GAT) – same GNN processes each data graph separately to produce data node embeddings
- GNN with attention over task graph – produces updated embeddings for labels and data nodes
- Compute the logits





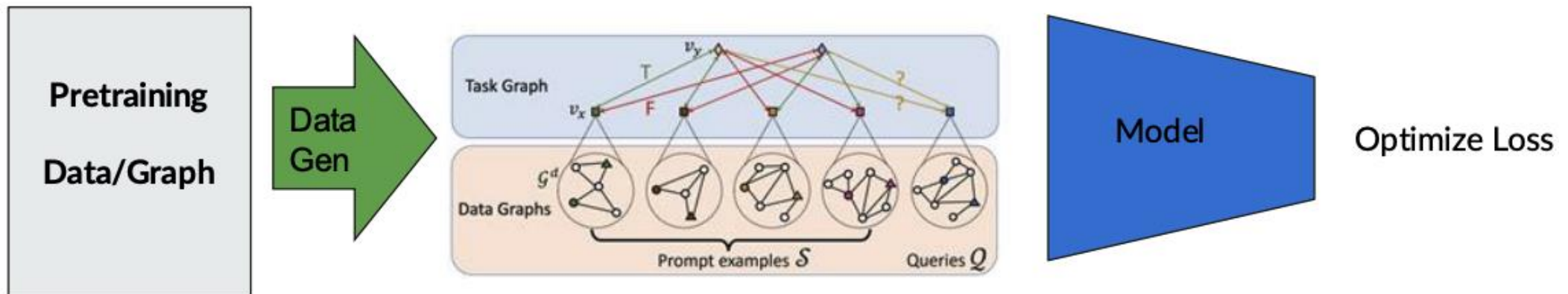
PRODIGY pretraining





In-context pretraining

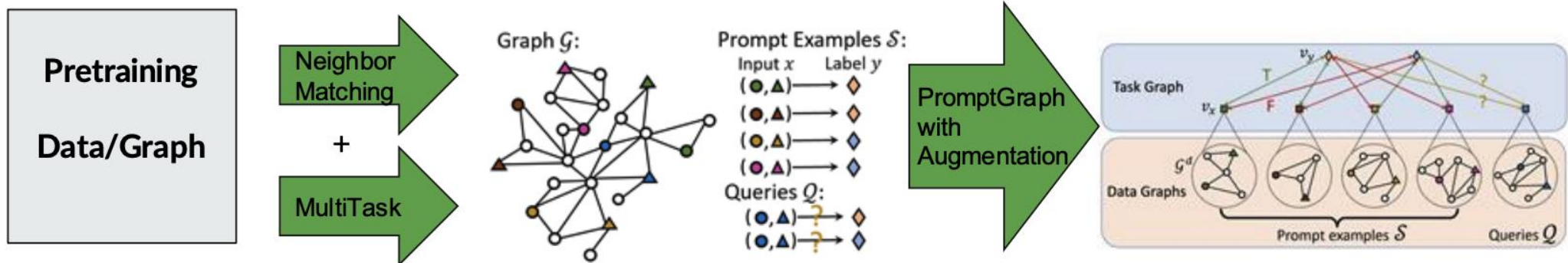
Goal is to create training examples represented in the Prompt Graph format that serve as a model pretraining dataset





In-context pretraining data generation

1. Few shot prompt generation

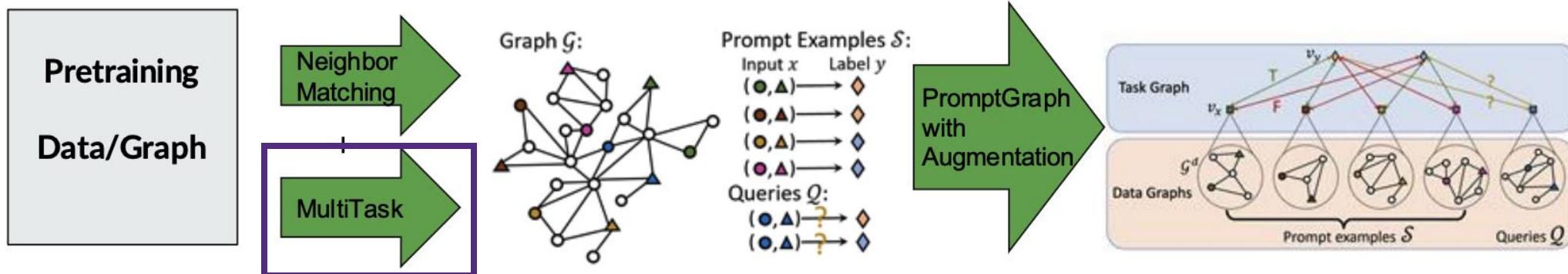




In-context pretraining data generation

1. Few shot prompt generation

- Supervised training on multiple tasks if labeled data is available



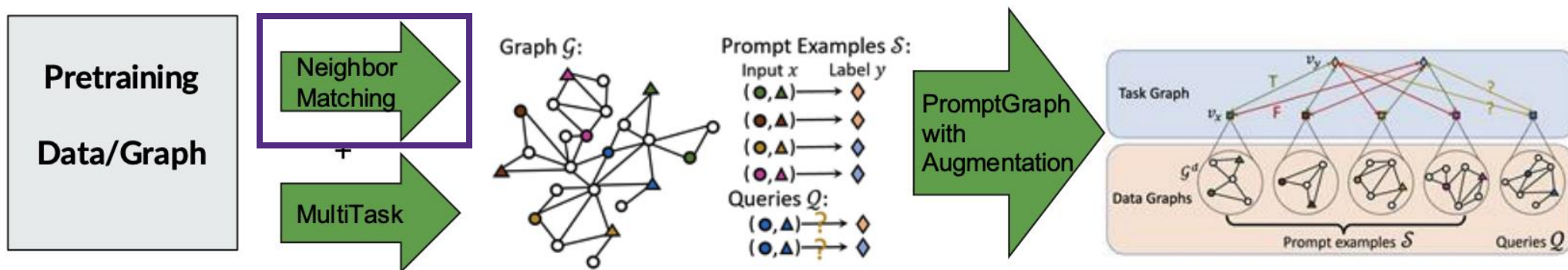


In-context pretraining data generation

1. Few shot prompt generation

- Supervised training on multiple tasks if labeled data is available
- Self-supervised prompts on neighbor matching task

How does this work?



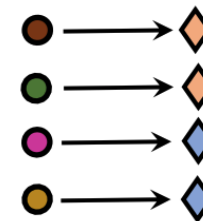
Neighborhood matching for pretraining

The task is to classify which neighborhood a node is in based on local structure (i.e., nodes within l -hops)

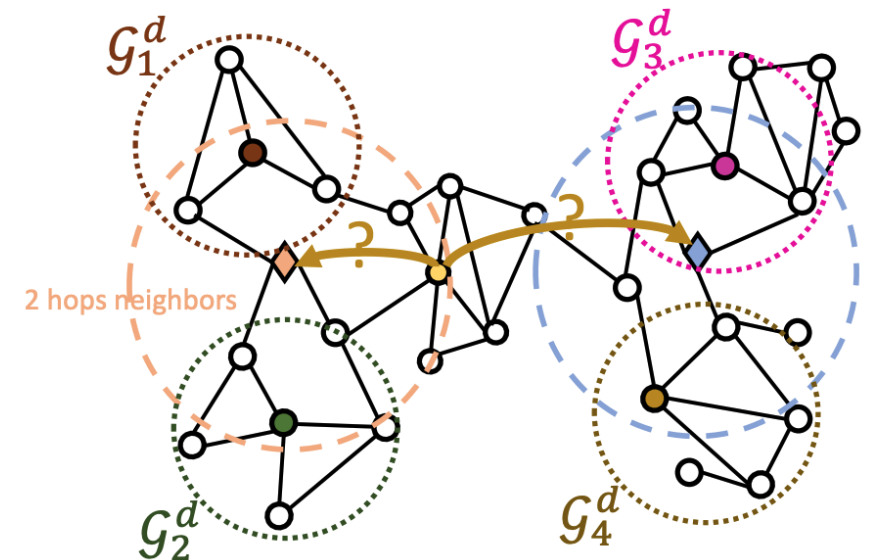
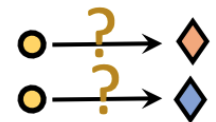
- Sample m nodes from the graph. For each node, v_i , assign the label c_i where $i \in [1, m]$
- For each node v_i :
 - Sample n nodes within l -hops and assign each of them label c_i
 - For each of these, construct the k -hop neighborhood
 - The adjacency matrix for the k -hop neighborhood is processed by the first GNN to form the data node for this prompt
- Repeat this process for sample queries

Prompt Examples S :

Input x Label y



Queries Q :





Pretraining loss function

Assuming both multi-task (MT) and neighbor matching (NM) prompts are included in training, loss function is

$$\mathcal{L} = \frac{1}{|Q_{NM}|} \sum_{i=1}^{Q_{NM}} CE(O_{NM,i}, y_{NM,i}) + \frac{1}{|Q_{MT}|} \sum_{i=1}^{Q_{MT}} CE(O_{MT,i}, y_{MT,i})$$

where:

$|Q_{NM}|$: number of queries in the neighbor matching tasks

$|Q_{MT}|$: number of queries in the multi-task tasks

$O_{NM,i} \in \mathbb{R}^m$: logits for query i from the NM task, m = number of class labels

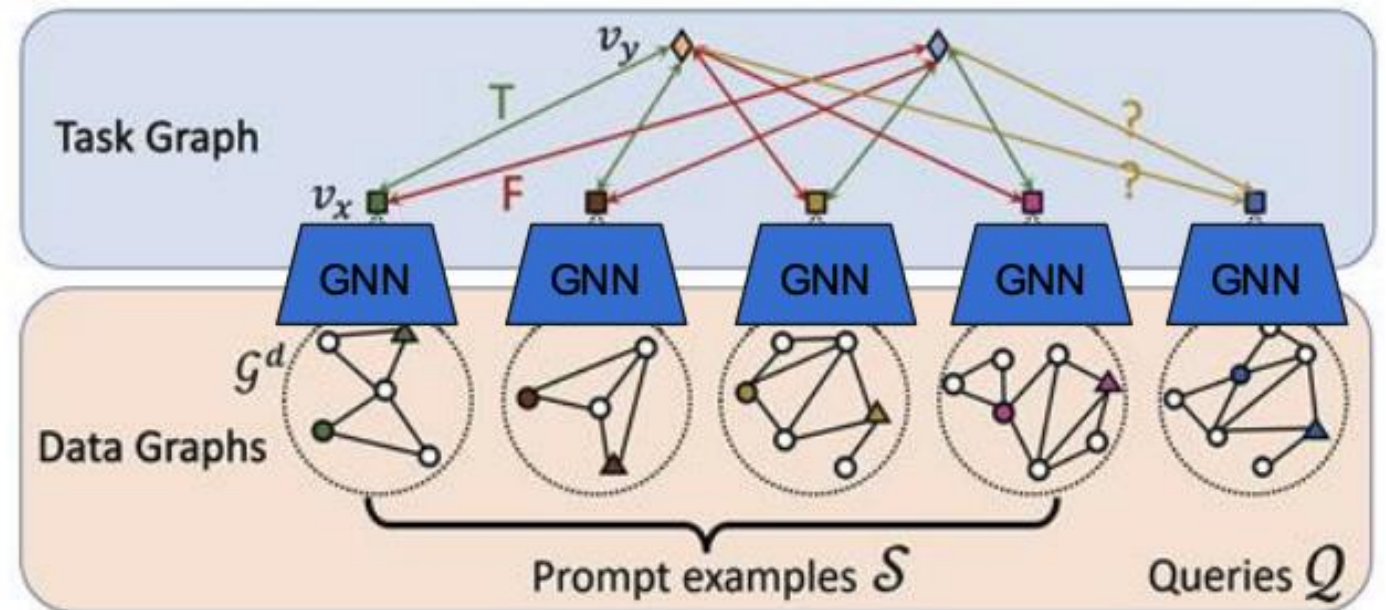
$O_{MT,i} \in \mathbb{R}^m$: logits for query i from the MT task, m = number of class labels

$y_{NM,i}$, $y_{MT,i}$: actual class labels

$$CE(O_{MT,i}, y_{MT,i}) = - \sum_{k=1}^m (y_{NM,i})_k \log [(O_{MT,i})_k]$$

An implementation note

- As formulated, the data graphs are processed by a GNN
 - Implies feature vectors are required for the nodes and any new nodes (e.g., for a new graph must have a feature vector from the same space)
 - Original method limited features to text descriptions which were first processed by a language model to form an embedding
 - Recent work has considered other task domains with consistent, but non-text features





Summary

- In-context learning models do not require additional parameter updates to perform new tasks
- Prompt Graph provides one possible formulation for representing few-shot prompts on graphs for diverse tasks
- Pretraining with neighborhood matching and multi-task learning enables few-shot learning
- Active area of research

